



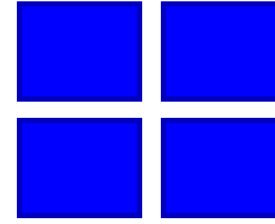
Ziel

**Optimierter Jitter für GEMM für verschiedene Input-Größen
und für transponierte Inputs und Outputs**

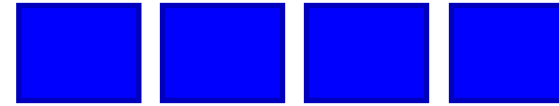


Ziel

- A und B ≥ 32 :

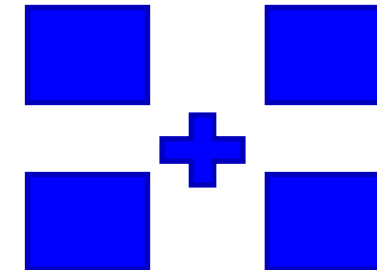


- A oder B ≥ 32 , anderer ≤ 16



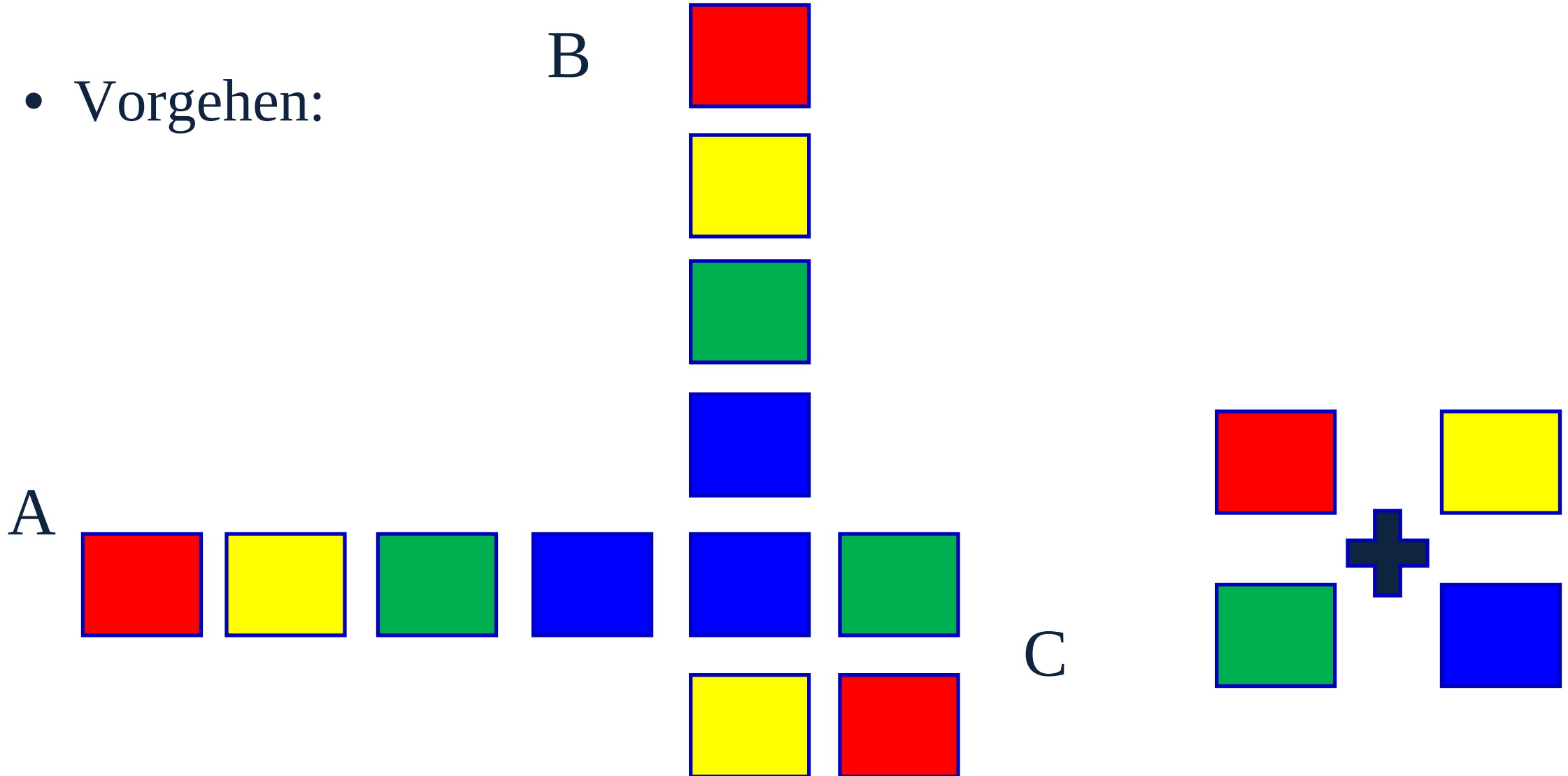
- A und B ≤ 16

ist es schneller, mehrere ZA-Tiles für k zu nutzen?



Multiple K-Tiles

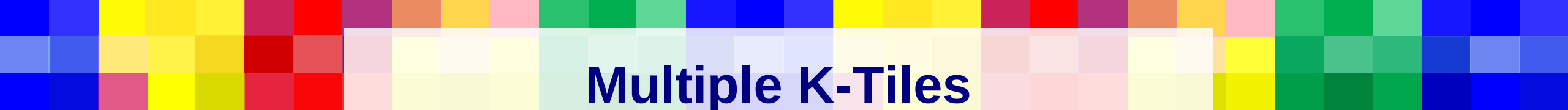
- Vorgehen:



Multiple K-Tiles

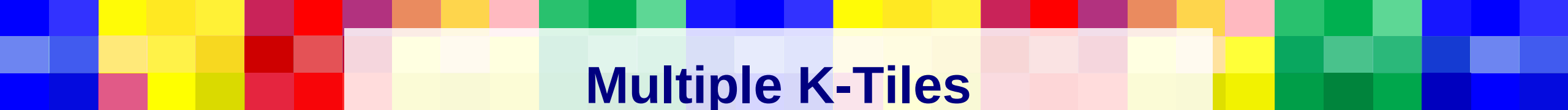
- Auslesen aus ZA Tiles
- addieren auf z Registern
- verschattet Speichern

```
.rept 4
movb { z0.s - z3.s }, za0h.s[w13, 0:3]
movb { z4.s - z7.s }, za1h.s[w13, 0:3]
movb { z8.s - z11.s }, za2h.s[w13, 0:3]
movb { z12.s - z15.s }, za3h.s[w13, 0:3]
fadd z0.s, z0.s, z4.s
fadd z1.s, z1.s, z5.s
fadd z2.s, z2.s, z6.s
fadd z3.s, z3.s, z7.s
fadd z0.s, z0.s, z8.s
fadd z1.s, z1.s, z9.s
fadd z2.s, z2.s, z10.s
fadd z3.s, z3.s, z11.s
fadd z0.s, z0.s, z12.s
fadd z1.s, z1.s, z13.s
fadd z2.s, z2.s, z14.s
fadd z3.s, z3.s, z15.s
st1w {z0.s - z3.s}, pn8, [x6]
add w13, w13, #4
add x6, x6, x10, lsl #2
.endr
```



Multiple K-Tiles

Ein K-Tile	471.8 GFlops
4 K-Tiles einfach	1146.6 GFlops



Multiple K-Tiles

- Addieren auf ZA Tiles

Multiple K-Tiles

fadd za.s[w8,0,VGx4], {z4.s-z7.s}

vectors = SVL / 8

vstride = vectors / 4

vec = (w8 + 0) % vstride

vectors = 64

vstride = 16

geschriebene Za Zeilen:

- w8 = 0 - ZAvector 0, 16, 32, 48
 - w8 = 4 - ZAvector 4, 20, 36, 52
 - w8 = 8 - ZAvector 8, 24, 40, 56
 - w8 = 12 - ZAvector 12, 28, 44, 60
-
- w8 = 0 - za0h.s slices 0, 4, 8, 12
 - w8 = 4 - za0h.s slices 1, 5, 9, 13
 - w8 = 8 - za0h.s slices 2, 6, 10, 14
 - w8 = 12 - za0h.s slices 3, 7, 11, 15

Multiple K-Tiles

- Auslesen aus ZA Tiles
- addieren auf ZA0 Tile
- verschattet Speichern
– bringt nichts

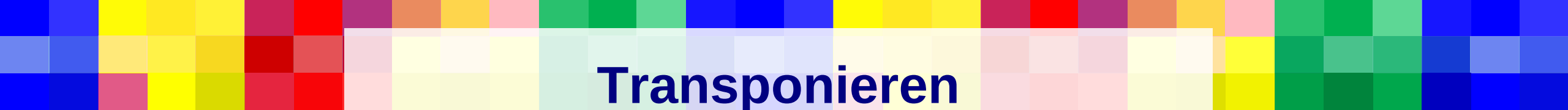
```
.rept 4
movb { z0.s - z3.s }, za.s[w8, 1, VGx4]
fadd za.s[w8,0,VGx4], {z0.s - z3.s}

movb { z4.s - z7.s }, za.s[w8, 2, VGx4]
fadd za.s[w8,0,VGx4], {z4.s - z7.s}

movb { z8.s - z11.s }, za.s[w8, 3, VGx4]
fadd za.s[w8,0,VGx4], {z8.s - z11.s}
add w8, w8, #4;
.endr
```

Multiple K-Tiles

Ein K-Tile	471.8 GFlops
4 K-Tiles einfach	1146.6 GFlops
4 K-Tiles sme2	1243.0 GFlops



Transponieren

- 3 Varianten:
 - ZA Array
 - TBL
 - SME2

Transponieren - ZA tile slices

- Matrix wird in horizontale ZA-tile-slices geladen
- Vertikale ZA-tile-slices enthalten Säulen der transponierten Matrix

```
ld1w za0h.s[w12, 0], p0/z, [x0]  
st1w za0v.s[w12, 0], p0, [x1]
```

	za0v.s[0]	za0v.s[1]	za0v.s[2]	za0v.s[3]
za0h.s[0]	0	1	2	3
za0h.s[1]	4	5	6	7
za0h.s[2]	8	9	A	B
za0h.s[3]	C	D	E	F

Transposition mittels ZA-tile-slices

Transponieren - TBL

- Transposition von 2x2 Teilmatrizen:

trn1 z16.s, z0.s, z1.s
trn2 z17.s, z0.s, z1.s

7	6	5	4	3	2	1	0	z0.s
F	E	D	C	B	A	9	8	z1.s

E	6	C	4	A	2	8	0	z16.s
F	7	D	5	B	3	9	1	z17.s

Transposition von 2x2 Teilmatrizen

Transponieren - TBL

- Transposition von 4x4 Teilmatrizen:

trn1 z16.d, z0.d, z2.d
trn1 z17.d, z1.d, z3.d
trn2 z18.d, z0.d, z2.d
trn2 z19.d, z1.d, z3.d

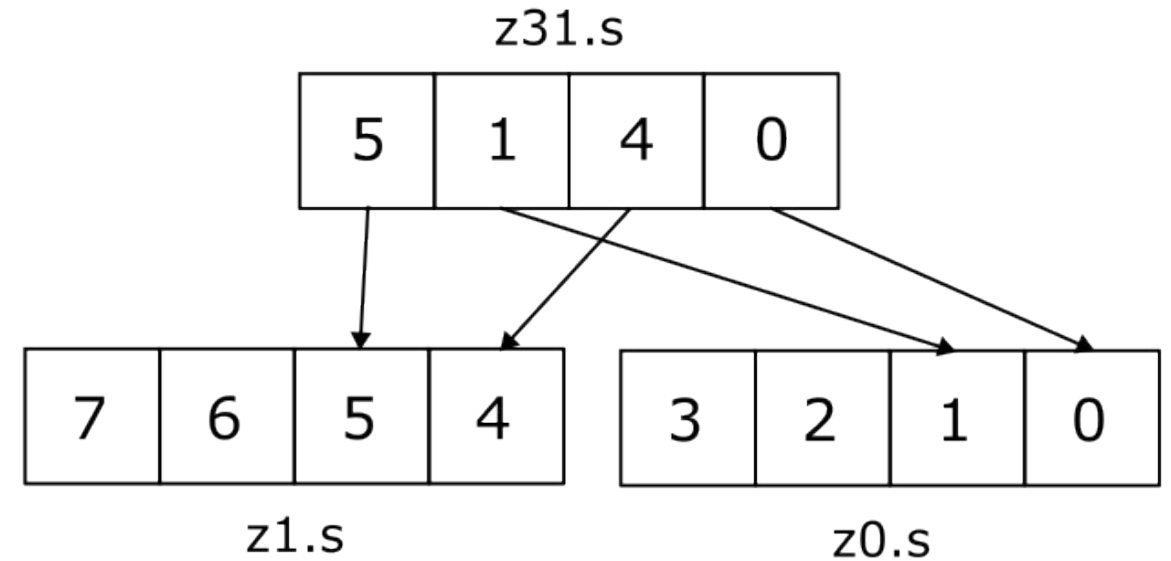
3	2	1	0	z0.s	9	8	1	0	z16.s
7	6	5	4	z1.s	D	C	5	4	z17.s
B	A	9	8	z2.s	B	A	3	2	z18.s
F	E	D	C	z3.s	F	E	7	6	z19.s

Transposition von 4x4 Teilmatrizen

Transponieren - TBL

- TBL Instruktion
 - Tabellenregister (z0, z1)
 - Indexregister (z31)
 - Indizes 0-15 bezeichnen z0, 16-31 bezeichnen z1

```
tbl z16.s, { z0.s, z1.s }, z31.s
```



TBL Instruktion



Transponieren - TBL

- Algorithmus
 - Transposition von 2x2 Teilmatrizen
 - Transposition von 4x4 Teilmatrizen
 - Transposition von 8x8 Teilmatrizen
 - Transposition von 16x16 Teilmatrizen

Transponieren SME2

- zip (four Registers)
- uzp
- uzp1.q (wäre gut)

```
zip { z0.d - z3.d }, { z0.d - z3.d }  
zip { z4.d - z7.d }, { z4.d - z7.d }  
zip { z8.d - z11.d }, { z8.d - z11.d }  
zip { z12.d - z15.d }, { z12.d - z15.d }
```

```
uzp { z16.s, z17.s}, z0.s, z4.s  
uzp { z18.s, z19.s}, z8.s, z12.s
```

```
uzp1 z26.d, z16.d, z18.d  
uzp2 z27.d, z16.d, z18.d
```

```
trn1 z28.d, z26.d, z27.d  
trn2 z30.d, z26.d, z27.d
```

Transponieren SME2

Zeile 0	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Zeile 1	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
Zeile 2	32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	47
Zeile 3	48	49	50	51	52	53	54	55	56	57	58	59	60	61	62	63



zip {z0.d - z3.d} (Ausschnitt: neues z0)

z0 (neu)	0	1	16	17	32	33	48	49	2	3	18	19	34	35	50	51
----------	---	---	----	----	----	----	----	----	---	---	----	----	----	----	----	----

Spalte 0

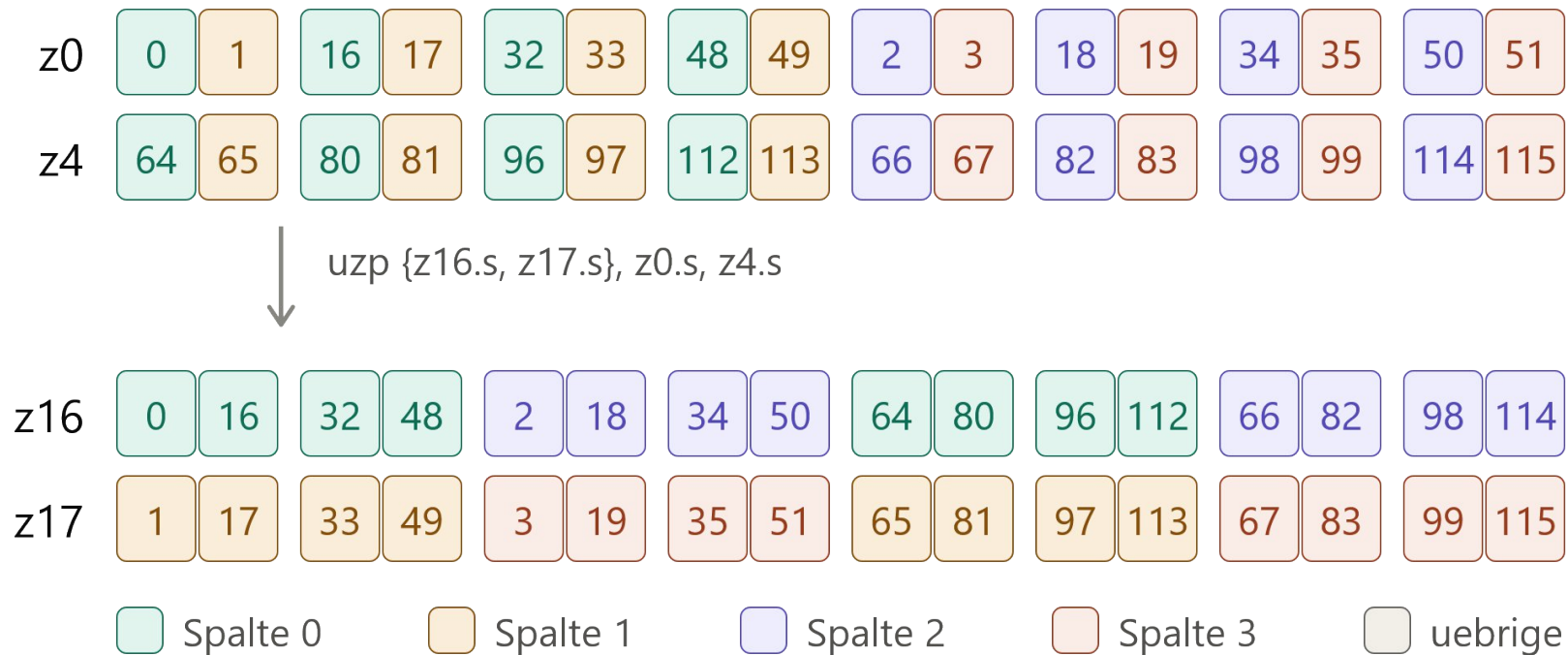
Spalte 1

Spalte 2

Spalte 3

uebrige

Transponieren SME2



Transponieren SME2

z16	0	16	32	48	2	18	34	50	64	80	96	112	66	82	98	114
z18	128	144	160	176	130	146	162	178	192	208	224	240	194	210	226	242



uzp1 / uzp2 z26/z27.d, z16, z18

z26 (uzp1)	0	16	2	18	64	80	66	82	128	144	130	146	192	208	194	210
z27 (uzp2)	32	48	34	50	96	112	98	114	160	176	162	178	224	240	226	242

 Spalte 0

 Spalte 1

 Spalte 2

 Spalte 3

 uebrige

Transponieren SME2

z26	0	16	2	18	64	80	66	82	128	144	130	146	192	208	194	210
z27	32	48	34	50	96	112	98	114	160	176	162	178	224	240	226	242

↓ trn1 / trn2 z28/z30.d, z26, z27

z28 = Spalte 0	0	16	32	48	64	80	96	112	128	144	160	176	192	208	224	240
z30 = Spalte 2	2	18	34	50	66	82	98	114	130	146	162	178	194	210	226	242

 Spalte 0  Spalte 1  Spalte 2  Spalte 3  uebrige

Transponieren Benchmarks

Simulation K-Loop

ZA Array	273.59 GiBs
TBL	103.999 GiBs
SME2	110.268 GiBs



Transponieren im Gemm

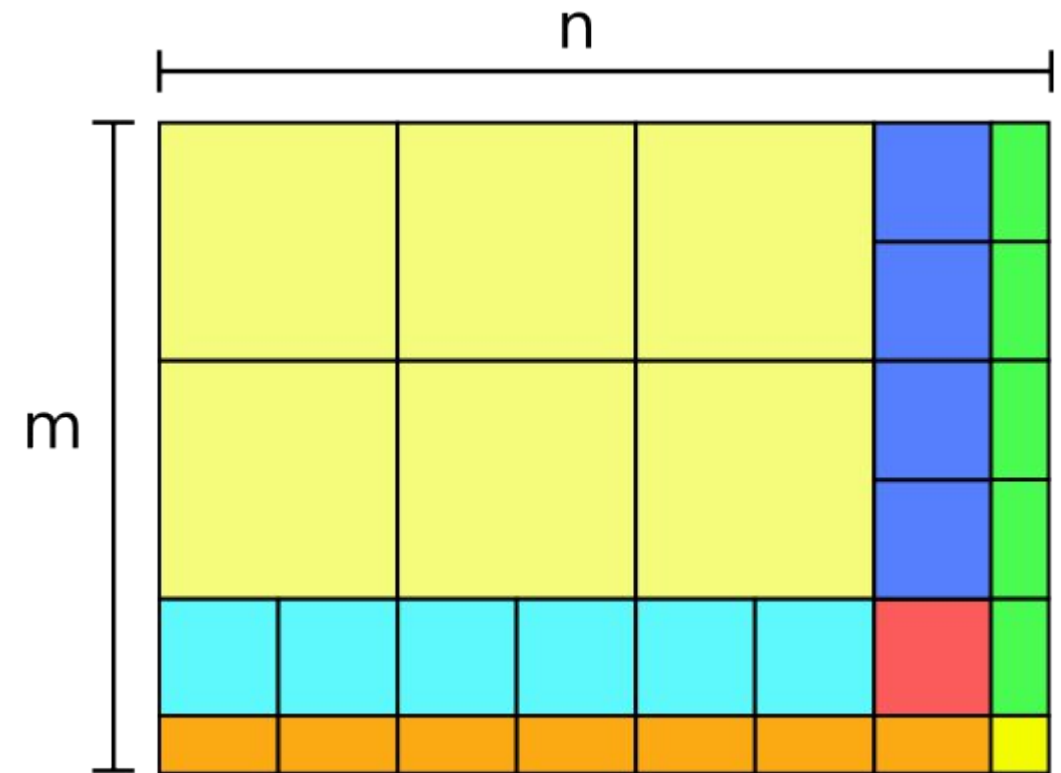
- ZA Tiles werden für fmopa gebraucht
- Tile müssen zwischengespeichert werden während Transponierung

Transponieren im Gemm

Kernel	GFlops
16x16x512 mul k	1243
16x16x512 mul k Trans: ZA Tile	475.4
16x16x512 mul k Trans: SME	464.3
512x512x512	1811
512x512x512 Trans: ZA Tile	652.5
512x512x512 Trans: SME	613.2

Jitter

- Jitter verwendet GEMM-Mikrokernels mit festen m/n - und flexiblen k -Dimensionen
- Algorithmus:
 - Berechnung eines Tilings der Ausgabematrix
 - Übersetzung der einzelnen Tiles in Schleifen über GEMM-Mikrokernels



Aufteilung einer Matrix $C=AB$

Jitter

m	n	k	trans_a	trans_b	trans_c	GFlops
512	512	512	0	0	0	463.532
512	512	512	0	0	1	462.618
512	512	512	0	1	0	1796.74
512	512	512	0	1	1	1766.05
512	512	512	1	0	0	293.548
512	512	512	1	0	1	293.298
512	512	512	1	1	0	443.519
512	512	512	1	1	1	442.374

Jitter

m	n	k	trans_a	trans_b	trans_c	GFlops
300	300	512	0	0	0	405.051
300	300	512	0	0	1	410.581
300	300	512	0	1	0	1221.12
300	300	512	0	1	1	1370.97
300	300	512	1	0	0	262.962
300	300	512	1	0	1	264.459
300	300	512	1	1	0	389.804
300	300	512	1	1	1	394.305